Handoff: Babbage's Hall

For a new chat: how to work, build, test, and what's next · 26 October 2026

Read this first in a new chat. It says how William likes to work, where everything is, how to build and test, what has been learned, and what to do next. The list of machines is `MACHINES.md`; the hall itself is described in `README.md`. This file is updated whenever a chat ends.

1. What this is

- **Babbage's Hall** (`demo/WyomingWill/BabbagesHall/`), a Victorian engineering hall in the Bullet Physics Playground (bpp, **github.com/bullet-physics-playground/bpp**, branch `master`): cast iron, brass, a glass roof, holding **machines that compute**. A separate museum from William's Euler's Gallery (`../EulersGallery/`), built on its machinery (separate worlds, sandboxes, walking with Tab).
- **Every machine can be 3D printed and built.** Each exhibit's parts are OpenSCAD sources (the single source for the simulation and the prints), a `make-stls.sh` makes the STL files, and its README lists what to buy and how to assemble it. What is printed is what is simulated: no shortcuts in the parts that compute, and anything idealised is said on the placard and in FINDINGS.
- **The walk gets more interesting as you go**, least interesting first, Babbage's Difference Engine at the far end (`MACHINES.md` ranks them).
- Built so far: **the Differential** (west 1), **Lehmer's number sieve** (east 1), **the Pascaline** (west 2), **the Domino Half Adder** (east 2), and the **Digi-Comp II's accumulator** (west 3; its other registers are next). Covered bays wait for Kelvin's tide predictor (east 3), the Differential Analyser (west 4) and the Difference Engine (east 4).
- William M. Spears (`williammspears@gmail.com`) owns the WyomingWill demos; Jakob (koppi) wrote bpp. Clocks are William's own separate collection.

2. How William likes to work

- **Deliver every change as a patch plus the git commands.** He applies patches in a checkout called `bpp-push`, pushes, then pulls in his installation checkout (which has `pull.ff only`; never apply patches there). Give the commands as below, and say if bpp must be rebuilt (only when C++ changed).

```
git pull
git am ~/Downloads/<name>.patch
git push
```

- **Every patch includes all its documents, PDFs too.** Every `.md` has a typeset `.pdf` beside it (`./make-pdfs.sh`, or `tools/md2pdf.py` for one); rebuild the PDF whenever its `.md` changes. Euler's Gallery and the Rec Room have README.pdfs made with `../ArtificialPhysics/tools/md2pdf.py`.
- **Before sending a patch:** check it applies to the current `origin/master` (`git fetch` first; a patch built on one he hasn't applied fails, as the first `MACHINES.md` patch did); no trailing spaces (`git diff --check`); `tools/check-all.sh` passes; the visuals looked at (`tools/shots.sh`). A re-sent patch gets a new file name, so a stale download can't be applied.

- Commits are authored `William M. Spears <williammspears@gmail.com>`, with the Co-Authored-By trailer for Claude. Stage only the files you changed.
- He looks at the exhibits in the GUI and reports what he sees with screenshots: take his reports seriously and reproduce them before answering, and measure when the pictures don't settle it.
- Plain explanations; measure before claiming; say what was and wasn't tested.
- **His bpp is built with Lua 5.2;** a cloud workspace's is usually 5.1. Write Lua that runs on both: no bare `setfenv`/`getfenv` (give `loadfile` its environment with `"t", _ENV` on 5.2, as the exhibits' table loaders do), no `goto`. `tools/check-lua52.sh` checks the one place this has bitten (the table loader) under a standalone `lua5.2`.

3. Where everything is

Path	What
`hall.lua`	The hall: architecture, the EXHIBITS list (walking order, placards, `floorY`), sandboxes, walking and camera flights, resting, the cost meter
`exhibit-table.lua`	The standard exhibit table (README "Exhibit standard")
`MACHINES.md`	All 47 machines considered, ranked by interest and scored for complexity, with how each would go in bpp and be printed; the rope test
`README.md`	The hall: bays, keys, how it works, the cost meter, the exhibit standard, notes for exhibit authors
`HANDOFF.md`	This file
`make-pdfs.sh`, `tools/md2pdf.py`	The PDFs
`make-labels.py`, `hall-meshes/`	The placards' lettering (Pillow, NumPy, DejaVu fonts)
`tools/check-all.sh`	Every exhibit's tour, on its own and in the hall
`tools/shots.sh`, `tools/keys.sh`	Screenshots on a virtual screen; with keys, hover, camera flights
`tools/check-lua52.sh`	The Lua 5.2 sandbox check
`tools/rope-test.lua`	The rope experiment (for the tide predictor)
`exhibits/DigiComp2/tests/toggle.lua`, `tests/switch.lua`	One Digi-Comp flip-flop alone (with its foot, done hole and clear lane), and one switch
`exhibits/<Name>/`	One folder per exhibit: its script, `parts/` (OpenSCAD), `make-stls.sh`, `README.md`/`.pdf`, `FINDINGS.md`/`.pdf`

Related: `../EulersGallery/` (the gallery the hall's machinery came from; its tables still follow its own sizes, see §7), `../RecRoom/` (games room: the cost meter and suspending came from there), `demo/claude/digicomp.lua` (an incomplete Digi-Comp II, a starting point), `module/scad/gearsv50.lua` and the Differential's `parts/gearsv50.scad` (bevel gears).

4. Building, running, testing

```
git clone https://github.com/bullet-physics-playground/bpp ~/bpp && cd ~/bpp
apt-get update
apt-get install -y libbullet-dev libluabind-dev liblua5.1-0-dev liblua5.2-dev \
  libqglviewer-dev-qt5 libsdl2-mixer-dev libassimp-dev qtbase5-dev libglew-dev \
  xvfb xdotool imagemagick openscad lua5.2 povray
```

```
pip install --break-system-packages trimesh manifold3d markdown playwright
mkdir -p build && cd build && qmake ../bpp.pro CONFIG+=release && make -j2 release
```

That gives `build/release/bpp` (a few minutes to ten). The tools find it there, in `release/`, or on the path; or set `BPP`.

- **Headless:** from the top folder, `build/release/bpp -f demo/WyomingWill/BabbagesHall/hall.lua -n 600`. `HALL_START=pascaline` starts at an exhibit. The exhibits run on their own the same way, each with a tour that prints `RESULT <name> tour: PASS`.
- **GUI on a virtual screen:** `tools/shots.sh` (see its header); `LIBGL_ALWAYS_SOFTWARE=1`, so drawing is slow and the simulation runs slower than real time there. In the GUI, `print` goes to bpp's console pane, not the terminal: write to a file (the meter's `HALL_METER_FILE`, the flights' `HALL_FLY_LOG=file`) to read it.
- **Printed parts:** each exhibit's `make-stls.sh` (OpenSCAD 2021.01 here; William's is newer: avoid `assign()` and `polyhedron(triangles=)`). Check every STL is one watertight solid (trimesh), and that parts don't overlap at rest (manifold3d); OpenSCAD renders pictures under `xvfb-run`.
- **POV-Ray:** `build/release/bpp -f ... -n 2 -e` writes `export/<script>/` (untracked; delete it after); render with `povray +I<script>.pov +K2 +W800 +H450 +L<bpp folder>`.

5. What has been learned (bpp and Bullet)

- **OpenSCAD parts** are cached by the text asked for, not by the files' contents: put a checksum of every file the part includes into the text (each exhibit's `checksum()`). Set `collides` before `v:add`, the transform after. A body's origin must be its centre of mass.
- **Two bpp processes building the same new part at once** can collide in the mesh cache: run tests one at a time after a part changes.
- **Gears:** mesh-on-mesh (GImpact) bevel teeth jam; a compound of convex hulls (one per tooth plus the body, from the same SCAD source) meshes well (`exhibits/Differential/FINDINGS.md`).
- **Hinge angles** (`getHingeAngle`) are clockwise-positive as seen down the hinge's axis, the opposite of a right-handed turn. A hinge's motor speed counts the same way as its angle (measured on the Digi-Comp's wheel).
- **A pin's friction** can be had from a hinge's motor held at no speed with a small impulse a step (`enableAngularMotor(true, 0, torque * step)`); the Digi-Comp's rockers use it, from PLA on PLA at the boss.
- **Over-centre parts** (a rocker whose centre of mass is above its pin) rest in a very shallow dip: with no friction they bounced back out of it, with pin friction they stop wherever they're left. Give them weight high up (the Digi-Comp's rockers have a steel ball pressed in) and test both extremes.
- **Walls as boxes** run past their ends by half their thickness if made `length + thickness` long, so the joints are solid: check where such an end meets a moving part (the Digi-Comp's feeder wheel rubbed two), or narrows a channel (two jammed the Digi-Comp's clear lane). Mitre a lane's walls at its bends (`laneWalls` in the Digi-Comp's geometry).
- **A ball pushing a part out of its way** pushes it only until it can squeeze past, which may be nowhere near where the part should end up: to push it all the way, guide the ball (the Digi-Comp's clear lane narrows beside each foot).
- **A part with holes** (the Digi-Comp's upper plate) is a static body of boxes round the holes, drawn not by them (`transparency = 1`) but by the OpenSCAD part. `transparency` near 0.35 draws a see-through plate well.

- **Luabind objects** from `v:eachContact` can't be compared with `==`; look them up as table keys (`names[obj]`).
- **Small steps:** the exhibits take 2400 steps a simulated second, stepped by hand in `preSim` (`common.setTiming(STEP, 0, STEP)`); the teeth and the dominoes need it. Sleeping is off for moving parts (state 4), so each exhibit **rests** (stops stepping) once nothing moves; it changes nothing computed.
- **Springs** that matter are simulated as the printed part: the Pascaline's detent is a body on a hinge turned back by the leaf's stiffness from PLA (3EI/L), and its working range of thickness was found by sweeping it.
- **Rope:** none built in; a rope of beads on ball joints works if the rope isn't far lighter than what hangs on it (`MACHINES.md`, "Rope").
- **Mouse:** scripts get `onHover` (after half a second still, no button down) but no clicks; keys override bpp's own one-letter keys.
- `btTransform` has no `getOrigin` in bpp's Lua; `.trans`, `:mult` and `getCenterOfMassTransform` are what there is.
- **Compounds:** a `btCompoundShape` in bpp's Lua takes only a few dozen children (each costs one of luabind's few dependency slots; past them bpp stops with an assertion in `object_rep.cpp`). Build big bodies as trees of small compounds, neighbours together (Lehmer's sieve's `tree()`), and don't add and remove children over and over: move them (`updateChildTransform`), as the sieve's pegs do.
- **Cylinders:** bpp's Lua has `btCylinderShapeX` and `btCylinderShapeZ` but no constructor for the upright `btCylinderShape`: stand a Z one up.
- **Convex pieces** from a 2D outline: `exhibits/LehmerSieve/parts/make-geometry.py` (shapely's constrained triangulation, merged while convex); check that the pieces' areas add up to the outline's. Corners inside an outline cost pieces; draw holes and inner arcs coarser for the contacts than for the print.
- **An OpenSCAD part made a body** must be set to collide before `v:add` (the exhibits' `part()` turns collisions off).
- `v:eachContact(fn)` calls `fn(a, b, x, y, z, nx, ny, nz, distance, impulse)` for every contact: the quickest way to find what jams.
- **Geneva drives** index well but their locking disc only bounds a star's turn (about a degree either way): a sprung detent per star puts it on its place, and the disc's relief must clear the star's lead and lag (`exhibits/LehmerSieve/FINDINGS.md`).
- `.gitignore` at the top ignores `*.scad`: the hall's own `.gitignore` has `!*.scad`, and STL files are not kept (each `make-stls.sh` makes them).

6. How to make an exhibit

1. Pick it from `MACHINES.md`; it goes in the bay its interest says (the walk rises), which may move others along (`hall.lua` EXHIBITS; README table; PDF subtitles in `make-pdfs.sh`).
2. Prototype the mechanism in a scratch script first and measure it.
3. `exhibits/<Name>/`: the script runs on its own and in the hall (find its folder with the `DIR` loop the others use; `hallPath` for OpenSCAD's full path), uses `exhibit-table.lua` (the machine's footprint), the start hold, resting, a tour that prints `RESULT ... PASS/FAIL`, sliders, keys, help text and hover text.
4. `parts/*.scad` as the single source of geometry (or, as the Pascaline, a Lua geometry file that writes the SCAD); `make-stls.sh` with each part turned to print; check the STLs and overlaps.
5. Sweep friction and speed, record it in `FINDINGS.md`; `README.md` with the parts table, what to buy, assembly; both PDFs; a `make-pdfs.sh` line.

6. In the hall: its EXHIBITS entry (`floorY = -100`), placard lettering (`make-labels.py`), `tools/check-all.sh` (add it there), screenshots.
7. **Make it show its working in the 3D view from the start**, not only in the console and tooltips: William watches in the GUI. Lehmer's sieve first showed nothing of N, x or the answer, and its probe's depth was hidden; it got a board of number drums (the exhibit's, said so) and a printed depth scale. When a machine runs on by itself, pause between steps so each reading can be seen (the sieve's "through" drum was cleared in the frame it was set). Say what the machine computes and what the script does for the operator (the sieve only says "maybe"; the square test is Lua's).

7. Where things stand and what's next

- **Done (2026-10-21): Lehmer's number sieve** (east 1): four Geneva stars of 9, 8, 7 and 5 places on one axle, driven from one shaft, detents to centre them, pegged holes, a cam-lifted probe; it factors 1241 (and 1679, 1157, 2813: `N`) by Fermat's way. **2026-10-25:** William found it hard to see what it computed: now a board beside it (the exhibit's) shows N, x, each wheel's hole under the probe (green open, red pegged), the depth, and at a stop x*x - N, the verdict and the factors; a printed depth scale on the probe's guide reads how far the probe went; `T` runs the tour for the number `N` chose. The wheels' colours and the see-through cam are still first guesses. 2026-10-26:* the depth now stays on the board between turns, and the camera is further back; William's window is narrower than the virtual screen (about 1450 px of 3D view, not 1658), so frame views with room to spare. He asked whether the cam is one print: it is (one watertight solid); only the simulation's contact shape is 72 convex wedges. Offered, not done: finer cam steps (5 degree flats now). It is the dearest exhibit to step (a 25 s tour took about three minutes here, headless): if it is slow on his computer, `LS_HZ=1200` halves that and still passes.
- **Done (2026-10-26): the Digi-Comp II, step 1, the counter** (west 3): three printed rockers, tipped over by the balls alone, and a feeder wheel. **The same day, step 2a, the accumulator,** which replaces it. William asked for the rest of the real machine; its manual (Evil Mad Scientist's copy, cdn.evilmadscientist.com/KitInstrux/DCII-manual.pdf) gives the registers: accumulator 7 flip-flops, memory 4 bits, MQ 3, distributor 3 (a ring counter), mode flip-flops T1 to T6 and complement CF1, CF2; switches MULTIPLY, CLEAR, COUNT, COMPLEMENT, OVERFLOW (run/halt) and AUTO/MANUAL; the ball's path INPUT, MULTIPLY, CLEAR, COUNT, then down the accumulator. Built now: a **two-level** board (balls that are done drop through holes to the lower level and roll to the collector), the 7-bit accumulator (each rocker has a foot on the lower level for CLEAR: a wedge a clear ball pushes back to 0), hand-set **switches** (flaps) for CLEAR and COUNT, and START (the feeder wheel, turned by hand). Keys: Space START, C and N the switches, 1 to 7 set a bit by hand. Board 322 x 878 mm, 10 upper and 10 lower tiles; table 100 x 150. Not yet seen by William.
- **Next: step 2b, add and multiply:** the memory (4 bits, read without being changed: in the original a 1 sends the ball into the accumulator at that bit, a 0 drops it through a hole), the MQ (3 flip-flops, counted down by one ball in four, which asks whether it was 0) and the distributor (3 flip-flops, a ring counter sending successive balls to M4, M2, M3, M1, then the MQ), and the MULTIPLY switch. Each new part alone first, in `tests/`. A memory bit entering the accumulator needs a way into each flip-flop's funnel from the side: the left half of the plate is free for them. Then **AUTO** (the START lever: see FINDINGS for why the first escapement failed) and **step 2c**, subtract and divide (the complement: T1 to T6, CF1, CF2, the COMPLEMENT switch; the manual describes its four-ball sequence). William pointed to a replica too: www.instructables.com/Digi-Comp-II-Replica/ (only its summary could be fetched here: the original's frame is about 14 x 28 inches).
- The other 39 machines in MACHINES.md have no bays: adding any grows the hall by a row of two, each going in by its rank (the Galton board, 13, and the planimeter, 14, would go in after the sieve,

before the Pascaline).

- **Then, in the order of the covered bays,** each its own step: **Kelvin's tide predictor** (east 3: cranks and a bead rope over pulleys), **the Differential Analyser** (west 4) and **the Difference Engine** (east 4). New bays (the hall grows by a row of two) would suit the Galton board and planimeter (quick and clear) or the Enigma and the Antikythera mechanism (striking, within reach).
- **Settled (2026-10-21):** William has tried the Pascaline (the mouse and the keys) and it works well; the stylus collision he thought he saw wasn't one. Euler's Gallery keeps its own tables as they are; the exhibit standard is the hall's. The Rec Room's camera flights look good, the top-down table view included.
- **Open, for later:** POV-Ray finds textures only with `+L` the bpp folder (their paths are relative to it): Jakob has it. A printed detent (`detent.stl`, 1.2 mm leaf) would test the stiffness worked out from PLA; William will print one some other time.

8. Opening message for the next chat

> Clone github.com/bullet-physics-playground/bpp and read demo/WyomingWill/BabbagesHall/HANDOFF.md, then MACHINES.md and README.md there. Build bpp as described, run tools/check-all.sh, and continue with HANDOFF.md section 7: [the next step, e.g. the Digi-Comp II's step 2b]. Deliver each step as a patch, with its documents and PDFs, and the git commands.